



***Animation ,Coloring
and transformation***

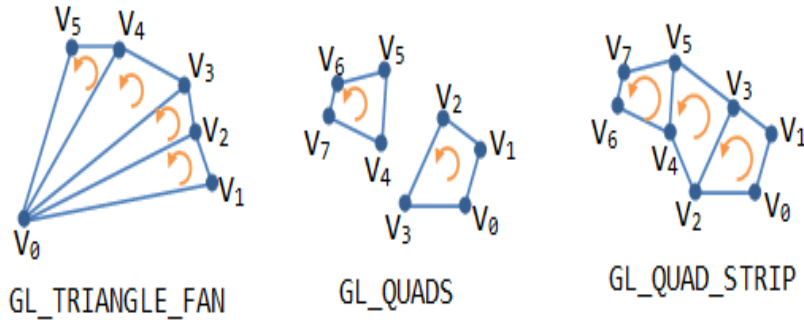
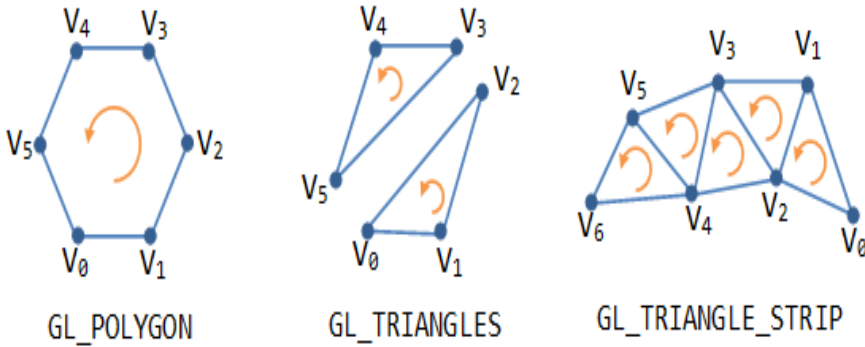
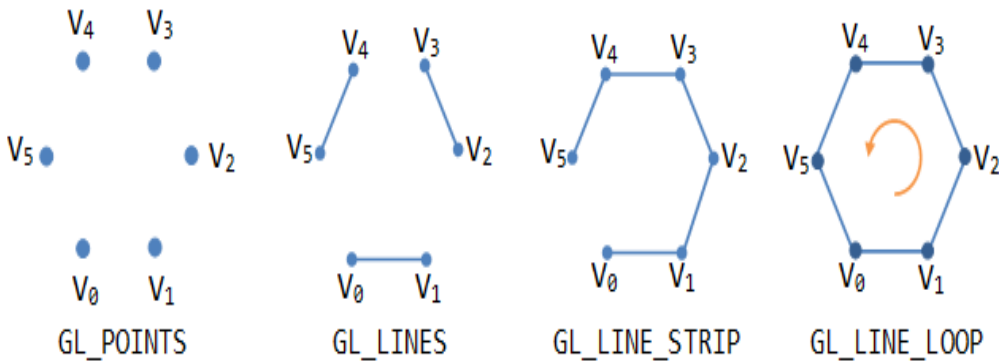
The constant `GL_POINTS` is built-into OpenGL. to draw other primitives, you replace `GL_POINTS` with `GL_LINES`, `GL_POLYGON`, etc.

`glColor3f()`
`glVertex2i(...)`

Table 1-1 : Command Suffixes and Argument Data Types

Suffix	Data Type	Typical Corresponding C-Language Type	OpenGL Type Definition
b	8-bit integer	signed char	GLbyte
s	16-bit integer	short	GLshort
i	32-bit integer	long	GLint, GLsizei
f	32-bit floating-point	float	GLfloat, GLclampf
d	64-bit floating-point	double	GLdouble, GLclampd
ub	8-bit unsigned integer	unsigned char	GLubyte, GLboolean
us	16-bit unsigned integer	unsigned short	GLushort
ui	32-bit unsigned integer	unsigned long	GLuint, GLenum, GLbitfield

In OpenGL, an object is made up of geometric primitives such as triangle, quad, line segment and point. A primitive is made up of one or more vertices. OpenGL supports the following primitives:



OpenGL Primitives

Animation

- Animation : is just **clear and redraw** in the successive frames.
- **60 fps (60 frame per second)** that means the screen will be refresh 60 times in one second

Animation

- `glutInitDisplayMode( GLUT_DOUBLE );`
 - this means you will use two frame buffers.
 - First buffer (Front Buffer):
 - Which is being displayed on the Screen.
 - Second buffer (Back Buffer) :
 - Which you use to draw it is not being displayed on the screen it is just in memory.
 - After finishing your drawing you can swap these buffers (this means the front buffer will become the back buffer and the back buffer will become the front buffer)

Animation

- Animation in the **double buffering mode** is just redraw and swap.
 - We just **redraw our scene** on the back buffer and then swap the buffer (**that means the back buffer will become the front buffer**).

Animation

- `glutSwapBuffers();`
 - to swap between two buffers (front and back)
- `glutTimerFunc(1000,timer,0);`
 - to redraw the scene
- `glutPostRedisplay();`
 - to redisplay the scene

Coloring

- `glClearColor(0,0,0,1);`
 - Colors are Red Green Blue Alpha (RGBA)
 - The 'A' (or alpha) specifies the **transparency (or opacity)** index, with value of **1** denotes opaque (**non-transparent** and cannot see-thru) and value of **0** denotes **total transparent**.
 - Set **background color for the window**.
 - The value of any color in range [0,1]
 - 0 means no value of this color
 - 1 means full color

Coloring

- `glColor3f(1,0,0);`
 - Set color for your object.
 - You can call this function before `glBegin` or after it.
- `glShadeModel(GL_SMOOTH);`
 - To change the type of shading
 - The default shading type is `smooth` which means blending will occur.
 - Set shading type to `flat` which means blending will not occur the color of object will be the first color .
 - It is usually better to keep the shading smooth.

transformation

- Matrices transformation:
 - Translation.
 - Rotation.
 - Scaling

Translation in 2D

- `glTranslatef(-10,5,0);`
 - `glLoadIdentity();`
 - Move translation to its default values at (0,0,0).
 - Is actually a type of **matrices transformation**.
 - Translation is actually **shifting of the origin**.
 - You **need to translate before** you draw.
 - Translation **will not alter the geometry** of your object.
 - Translation **change the position** at which the object is drawn.
 - Translation will help you **draw the same object at different positions without altering** its geometry.

Translation in 2D

- Take note that `glTranslatef` function must be placed **outside the glBegin/glEnd**, where as `glColor` can be placed **inside glBegin/glEnd**.